



// session 04 beginner Python

Write the flight plan

Use a loop to fly through every world in your galaxy.



Three worlds. Which one is at index **1**?

0

Verdantis

1

Kragnor

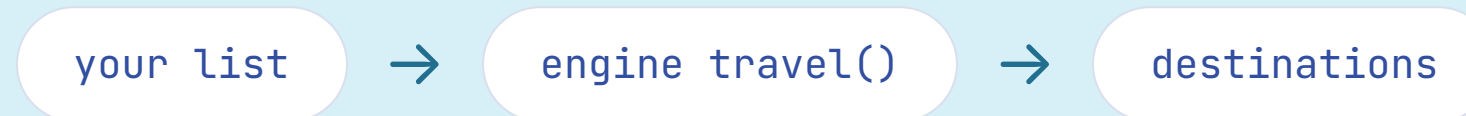
2

Aquifera

// the problem borrowed movement



Your list sets the route. The engine still flies it.



A for loop repeats once for each value.

Python takes the next value, runs the indented block, then comes back for another.



range(5) supplies five numbers.

Before running it, write the output you expect.

```
for i in range(5):  
    print(i)
```



// predict where does range stop?

**What is the final number printed
by range(5)?**

4

5

6

```
// reveal start included, stop excluded
```



0, 1, 2, 3, 4

Five values. The stopping number is not included.

`range(5)` → **indexes 0 through 4**

One index for every planet.

`len(planets)` supplies the stopping number. `planets[i]` reads that position.

```
planets = ["Verdantis", "Kragnor", "Aquifera"]

for i in range(len(planets)):
    print(f"Planet {i}: {planets[i]}")
```




```
// predict second iteration
```

When `i` is **1**, what does
`planets[i]` give us?

Verdantis

Kragnor

Aquifera

while repeats while a condition is true.

Useful when the number of repeats is unknown. We will not use it for today's game loop.

```
count = 0
while count < 3:
    print(count)
    count = count + 1
```

Each loop pass has six jobs.

1 spend oxygen

2 show destination

3 ask the player

4 process choice

5 show status

6 check defeat

Import the building blocks.

The engine still draws screens and calculates encounters. You control when each block runs.

```
from engine.display import (  
    show_intro,  
    show_destination,  
    show_encounter,  
    show_status,  
    show_victory,  
    show_defeat,  
)  
from engine.galaxy import create_galaxy  
from engine.encounters import process_encounter, process_water_planet  
import constants
```

Start with three pieces of state.

Create the galaxy, then copy the starting resources into variables inside `main()`.

galaxy

ordered destinations

oxygen

starts from constants

hull

starts from constants

01 / inside main

Reach every destination.

Replace `travel()` with this partial loop, then run it.

```
for i in range(len(galaxy)):
    destination = galaxy[i]
    show_destination(destination, i, len(galaxy))
```

02 / inside main

Each jump costs 8 oxygen.

Add the subtraction inside the loop: 30 → 22 → 14.

```
for i in range(len(galaxy)):
    destination = galaxy[i]
    oxygen = oxygen - 8
    show_destination(destination, i, len(galaxy))
```


// supplied pattern engine labels



destination["has_water"]

Read this as: “look up the destination’s **has water** label.”

True

process the water bonus

False

skip that block

03 / inside loop

Stop or fly past.

The engine returns updated resources and narration. Catch all three values.

```
if should_stop():
    oxygen, hull, narration = process_encounter(destination, oxygen, hull)
    show_encounter(narration)
    if destination["has_water"]:
        oxygen, hull, water_narration = process_water_planet(oxygen, hull)
        show_encounter(water_narration)
else:
    show_encounter(" You fly past without stopping.")
```

04 / inside loop

Stop when a resource reaches zero.

Each matching check shows the reason, then return ends main().

```
if oxygen ≤ 0:
    show_defeat(constants.SHIP_NAME, "Oxygen depleted")
    return
if hull ≤ 0:
    show_defeat(constants.SHIP_NAME, "Hull destroyed")
    return
```

Put the passes inside one for loop.

Use the supplied full reference beside your editor. Verify each checkpoint before moving down.

A destination + jump cost

B choice + encounter

C status + defeat

D victory after the loop

Key boundary: `show_victory(...)` is not indented inside the loop.

Replace `travel()` with your loop.

- 1 Update the engine imports.
- 2 Create oxygen and hull state.
- 3 Assemble the four tested passes.
- 4 Run `uv run game.py`.

Success looks like

Your code visits every world, obeys choices, detects defeat, and can reach victory.

30-35 min

Exercise every ending you claim works.

ROUTE 1

Skip all

Oxygen falls by 8 each jump.

ROUTE 2

Stop at water

The water message appears.

ROUTE 3

Lose oxygen

The loop ends immediately.

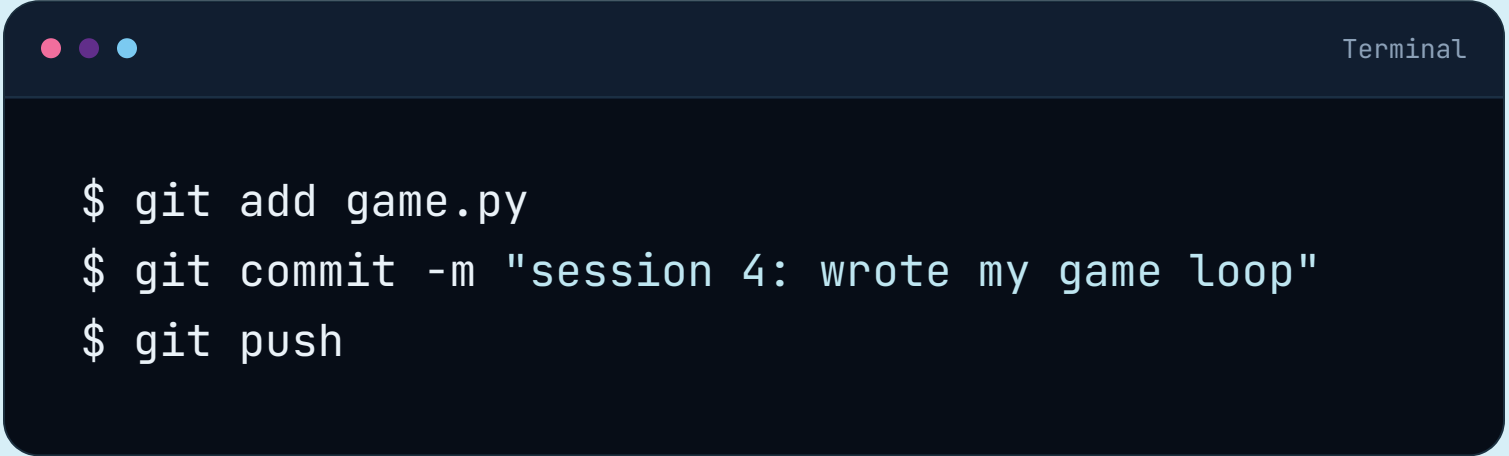
ROUTE 4

Reach the end

Victory appears once.

The game loop is yours.

Before pushing: explain why `range(len(galaxy))` gives exactly one pass per destination.



```
$ git add game.py  
$ git commit -m "session 4: wrote my game loop"  
$ git push
```

Tune the journey.

Choose one only after the working loop is pushed.

EASY

Change jump cost

Find values that feel easy or impossible.

MEDIUM

Count stops

Add one when the player stops.

PREVIEW

Replay prompt

Explore while without replacing the core loop.



// mission complete session 04

Your code now controls the journey.

Next: give each world names, descriptions, danger, and water.

Session 05 • Dictionaries